

Implementation and Analysis of Federated Personalized Contrastive Learning for Recommendation (FedPCL): A Reproducibility and Extension Study

Saumya Suman

IIIT SriCity john.ssuman@gmail.com

ABSTRACT

We present a complete stage-by-stage re-implementation of FedPCL – Federated Personalized Contrastive Learning for Recommendation [5] – without access to the original source code. The pipeline spans five cumulative stages: centralized LightGCN, FedAvg, clustering-based personalization, structural contrastive learning, and Local Differential Privacy. Experiments across four benchmarks (ML-100K, ML-1M, Steam, Amazon Electronics) show that our FedAvg baseline matches or exceeds the paper on three datasets, confirmed over 5 random seeds. The full Stage 5 implementation achieves near-paper performance on Steam (HR@10: 79.32% vs. 80.36%) and Amazon (HR@10: 31.78% vs. 34.04%), but significant gaps remain on ML-100K and ML-1M. A root-cause analysis identifies three correctable factors: a contrastive temperature deviation ($\tau=0.2$ vs. paper’s $\tau=0.3$), insufficient training rounds for ML-1M, and a preprocessing mismatch on Amazon. A partial ablation study is presented with a clear plan for completing the remaining entries.

I. INTRODUCTION

Federated recommendation systems must simultaneously address three competing objectives: *personalization* – each user has unique preferences that should be captured without centralizing raw data; *privacy* – interaction logs must remain on the user’s device; and *generalization* – despite local training, the global model must still benefit from cross-user patterns.

FedPCL [5] addresses all three within a unified framework. K-means clustering in the user embedding space groups users with similar tastes, enabling cluster-level personalized item tables without sharing raw interaction data. Structural contrastive learning, applied locally on an expanded GNN subgraph that includes 2-hop neighbour users, improves the quality of item and user representations. Local Differential Privacy (LDP) applied to uploaded gradients provides formal privacy guarantees quantified by a privacy budget ϵ .

This work documents a ground-up re-implementation of FedPCL without access to the original source code. The primary goals are: (i) verify that the algorithm as described in the paper is sufficiently specified to repro-

duce its results; (ii) identify and diagnose any performance gaps between our implementation and the paper’s reported numbers; (iii) propose a concrete experimental plan for closing those gaps.

The rest of the paper is organized as follows. Section 2 presents the complete FedPCL algorithm with all five stages and a pseudocode summary. Section 3 describes our computing environment and its significance for reproducibility. Section 4 covers experimental setup including dataset preprocessing strategies. Section 5 presents main results, multi-seed validation of the FedAvg baseline, and per-dataset analysis. Section 6 analyzes training convergence. Section 7 presents the ablation study. Section 8 discusses the LDP privacy-utility trade-off. Section 9 presents KDE embedding visualizations. Section 10 consolidates root causes. Section 11 outlines proposed improvements.

II. FEDPCL ALGORITHM

A. Architecture and Notation

Let $\mathcal{U}$ and $\mathcal{V}$ denote the user and item sets. Each client $u \in \mathcal{U}$ holds only its own interaction set $\mathcal{I}_u = \{i : \text{user } u \text{ interacted with item } i\}$, which never leaves the device. The server maintains three types of shared parameters:

- $\mathbf{E}_{\text{global}} \in \mathbb{R}^{|\mathcal{V}| \times d}$: the global item embedding table, derived each round as the simple mean of all cluster tables (paper Eq. 13). Initialized from Xavier uniform distribution.
- $\{\mathbf{E}_k\}_{k=1}^K$: per-cluster item embedding tables. Each $\mathbf{E}_k$ starts as a copy of $\mathbf{E}_{\text{global}}$ and is updated only from clients assigned to cluster k , allowing it to specialize to that cluster’s preferences over time.
- $\mathbf{c}_u \in \mathbb{R}^d$: the user embedding, kept *locally* on each client and updated via Adam. It is shared with the server *only* for K-means cluster assignment (with LDP noise applied in Stage 5). It is never used by the server for recommendation.

Each client runs LightGCN [1] on its *local subgraph*: a bipartite graph between the client’s user node and the

items in $\mathcal{I}_u$. In Stage 4, this subgraph is expanded to include 2-hop neighbour users (users who share at least one item with the anchor user) and their items, following the FedGNN graph-expansion approach [6].

B. Training Stages

Stage 1 – Centralized LightGCN. The full user-item bipartite graph is available on one machine. LightGCN propagation at layer k :

$$\mathbf{e}_u^{(k+1)} = \sum_{i \in \mathcal{N}_u} \frac{\mathbf{e}_i^{(k)}}{\sqrt{|\mathcal{N}_u| |\mathcal{N}_i|}}, \quad \mathbf{e}_i^{(k+1)} = \sum_{u \in \mathcal{N}_i} \frac{\mathbf{e}_u^{(k)}}{\sqrt{|\mathcal{N}_u| |\mathcal{N}_i|}} \quad (1)$$

The final representation is the mean across all $K+1$ layers: $\bar{\mathbf{e}} = \frac{1}{K+1} \sum_{k=0}^K \mathbf{e}^{(k)}$. Prediction: $\hat{y}_{ui} = \bar{\mathbf{e}}_u^\top \bar{\mathbf{e}}_i$. Trained with BPR loss [3]. This stage provides the centralized upper bound.

Stage 2 – Federated Averaging (FedAvg). Each selected client performs E local epochs on its own subgraph using BPR loss with SGD for item embeddings and Adam for the user embedding. The client computes the item delta $\Delta_u = \mathbf{E}_{\text{local}} - \mathbf{E}_{\text{global}}$ (sparse: only rows for items in $\mathcal{I}_u$ are non-zero) and uploads it to the server. The server performs weighted aggregation:

$$\mathbf{E}_{\text{global}} \leftarrow \mathbf{E}_{\text{global}} + \sum_{u \in S_t} \frac{|\mathcal{I}_u|}{\sum_{v \in S_t} |\mathcal{I}_v|} \Delta_u \quad (2)$$

where S_t is the set of 128 clients selected in round t . The user embedding $\mathbf{c}_u$ is updated locally and not shared.

Stage 3 – Clustering-Based Personalization. Every 10 rounds, the server collects all user embeddings $\{\mathbf{c}_u\}_{u \in \mathcal{U}}$ and runs K-means ($K=5$) to assign each user to a cluster based on their preference similarity. Each client then receives a personalized item table:

$$\mathbf{E}_{\text{personal}} = \mu_1 \mathbf{E}_{k(u)} + \mu_2 \mathbf{E}_{\text{global}}, \quad \mu_1 = \mu_2 = 0.5 \quad (3)$$

The client trains locally on $\mathbf{E}_{\text{personal}}$ and reports deltas; the server applies them to $\mathbf{E}_{k(u)}$ (cluster- k clients only), then recomputes $\mathbf{E}_{\text{global}} = \frac{1}{K} \sum_{k=1}^K \mathbf{E}_k$ (paper Eq. 13), ensuring the global model is always a balanced average of cluster specializations.

Stage 4 – Structural Contrastive Learning. The local subgraph is expanded to include up to `max_neigh` 2-hop neighbour users (ranked by number of shared items with the anchor user) and up to `max_items_neigh` items per neighbour. LightGCN runs on this expanded subgraph; at each even layer, neighbour user embeddings contribute to the anchor user’s updated representation, enabling richer collaborative filtering signals.

The per-client training objective adds two structural contrastive losses to BPR:

$$\mathcal{L} = \mathcal{L}_{\text{BPR}} + \beta_1 (\mathcal{L}_{\text{Con}}^U + \lambda \mathcal{L}_{\text{Con}}^V) + \beta_2 \|\Theta\|^2 \quad (4)$$

The *user-side contrastive loss* treats the even-layer output of the anchor user as a query, its layer-0 embedding as the positive key, and the layer-0 embeddings of neighbour users as negative keys (InfoNCE formulation):

$$\mathcal{L}_{\text{Con}}^U = -\log \frac{\exp(\mathbf{e}_u^{(l)} \cdot \mathbf{e}_u^{(0)} / \tau)}{\sum_{j \in \mathcal{U}_{\text{local}}} \exp(\mathbf{e}_u^{(l)} \cdot \mathbf{e}_j^{(0)} / \tau)} \quad (5)$$

where l is the highest even GNN layer ($l=2$ for $K=2$) and $\mathcal{U}_{\text{local}}$ is the anchor plus all neighbour users in the expanded subgraph.

The *item-side contrastive loss* uses cross-layer contrasting [5]: for each item, its even-layer GNN output is the positive sample and its layer-0 embedding is the reference key:

$$\mathcal{L}_{\text{Con}}^V = \text{CE}(\hat{\mathbf{E}}_p^{(l)} \hat{\mathbf{E}}_p^{(0)\top} / \tau, \mathbf{I}) \quad (6)$$

where $\hat{\mathbf{E}}_p^{(l)}$ and $\hat{\mathbf{E}}_p^{(0)}$ are the L2-normalised even-layer and layer-0 item embedding matrices respectively, and $\mathbf{I}$ is the identity target (each item matches itself). A 20-round warmup period disables the contrastive terms ($\beta_1=0$ for $t \leq 20$) to allow BPR to establish a stable signal first.

Stage 5 – Local Differential Privacy. After completing local training, each client applies LDP to both the item delta Δ_u and the user embedding $\mathbf{c}_u$ before uploading:

$$\tilde{g} = \text{clip}(g, \sigma) + \text{Laplace}(0, \lambda_{\text{Lap}}), \quad \varepsilon = \sigma / \lambda_{\text{Lap}} \quad (7)$$

Per-coordinate clipping to $[-\sigma, \sigma]$ bounds the ℓ_∞ sensitivity; Laplacian noise then provides ε -Local Differential Privacy. Note that the Adam optimizer state (first and second moment estimates for $\mathbf{c}_u$) remains entirely private on the client device – only the final noisy embedding value is shared.

C. Full Algorithm

Algorithm 1 summarizes the complete FedPCL procedure as implemented in our Stage 5. The key implementation detail is line 15: the global item table is not updated by direct FedAvg from all clients (as a naive reading of the paper might suggest), but is instead *recomputed* as the mean of the K cluster tables after those have been updated, faithfully implementing paper Eq. 13.

Algorithm 1 FedPCL – Server and Client Procedure (Stage 5)

Require: Rounds T , local epochs E , clusters K , warmup $w=20$

- 1: Server initializes $\mathbf{E}_{\text{global}}$, $\{\mathbf{E}_k\}_{k=1}^K$ (Xavier); clients init $\mathbf{c}_u$
- 2: Server runs initial K-means on $\{\mathbf{c}_u\}$
- 3: **for** round $t = 1, \dots, T$ **do**
- 4: Server selects subset S_t ($|S_t| = 128$)
- 5: **for all** client $u \in S_t$ **do**
- 6: Receive $\mathbf{E}_p^u = \mu_1 \mathbf{E}_{k(u)} + \mu_2 \mathbf{E}_{\text{global}}$
- 7: Expand subgraph to 2-hop neighbours
- 8: **for** epoch $e = 1, \dots, E$ **do**
- 9: Run LightGCN on expanded subgraph
- 10: $\mathcal{L} \leftarrow \mathcal{L}_{\text{BPR}} + \mathbf{1}[t > w] \cdot \beta_1 (\mathcal{L}_{\text{Con}}^U + \lambda \mathcal{L}_{\text{Con}}^V) + \beta_2 \|\Theta\|^2$
- 11: Update $\mathbf{E}_{\text{local}}$ (SGD), $\mathbf{c}_u$ (Adam)
- 12: **end for**
- 13: $\Delta_u \leftarrow \mathbf{E}_{\text{local}} - \mathbf{E}_p^u$; apply LDP (Eq. 7)
- 14: Upload $\tilde{\Delta}_u, \tilde{\mathbf{c}}_u$
- 15: **end for**
- 16: Update $\mathbf{E}_k$ from cluster- k clients only (Eq. 12)
- 17: $\mathbf{E}_{\text{global}} \leftarrow \frac{1}{K} \sum_{k=1}^K \mathbf{E}_k$ (Eq. 13)
- 18: **if** $t \bmod 10 = 0$ **then** Re-run K-means on all $\{\tilde{\mathbf{c}}_u\}$
- 19: **end if**
- 20: **end for**

III. COMPUTING ENVIRONMENT

All experiments were run on the hardware and software in Table 1. Documenting the precise system is essential for reproducibility: training time, peak memory usage, floating-point behavior, and GPU driver non-determinism all depend on the specific platform.

Table 1: System Specifications and Stage 5 Training Times

Component	Specification
CPU	Intel Core i5-13450HX (8P + 8E cores, 16 threads)
RAM	8 GB DDR4
GPU	NVIDIA GeForce RTX 3050 (6 GB VRAM)
OS	Ubuntu 22.04.5 LTS
Python	3.10.12
PyTorch	2.10.0 + CUDA 12.8
scikit-learn	1.7.2
<i>Stage 5 training time (400 rounds, measured)</i>	
ML-100K	116.2 min (≈ 17.4 s/round)
Steam	74.5 min (≈ 10.9 s/round)
ML-1M	≈ 4 hours
Amazon	≈ 40 min

Significance for reproducibility. Training is GPU-accelerated via CUDA 12.8. The RTX 3050 6 GB VRAM is sufficient for all datasets: the largest item embedding table (ML-1M: 3706 items $\times$ 64 dims $\times$ 4 bytes = 0.9 MB) plus five cluster tables (< 5 MB total) fit well within VRAM. The 8 GB system RAM comfortably holds all datasets in memory.

Run-to-run variance from cuBLAS non-determinism (atomic floating-point operations that produce slightly different results across runs) is empirically ± 0.2 – 0.5% on

HR@10, as measured in our 5-seed study. This is within the expected noise floor for this type of experiment. Because this variance is non-negligible relative to some of our measured gaps, all results should ideally be reported as mean $\pm$ std over multiple seeds for a final paper submission.

Training time logging. Each training script saves the total wall-clock time and average per-round time in the JSON log file. This allows training duration to be precisely reported and compared across datasets and stages.

IV. EXPERIMENTAL SETUP**A. Datasets and Statistics**

Table 2 shows statistics for all four datasets, including interaction density and the percentage deviation of our interaction count from the paper’s reported value.

Table 2: Dataset statistics. Interactions in thousands (K). $\Delta\%$ = relative deviation of our total from the paper’s count.

DS		Users	Items	Inter.(K)	Den.%	$\Delta\%$
ML-100K	Ours	943	1,682	100	6.30	0.0
	Paper	943	1,682	100	6.30	
ML-1M	Ours	6,040	3,706	1,000	4.47	0.0
	Paper	6,040	3,706	1,000	4.47	
Steam	Ours	3,757	5,113	115.1	0.60	+0.4
	Paper	3,753	5,134	114.7	0.60	
Amazon	Ours	1,504	1,954	39.1	1.33	+8.9
	Paper	1,435	1,522	35.9	1.65	

ML-100K and ML-1M reproduce the paper’s statistics exactly. Steam has a +0.4% interaction excess, which is negligible. Amazon has a +8.9% interaction excess and 28% more items (1,954 vs. 1,522), which is the primary dataset-level issue for that benchmark and is investigated in Section 4.2.

B. Dataset Preprocessing

ML-100K and ML-1M. Raw rating files with explicit timestamps. All ratings ≥ 1 are retained (implicit positive feedback interpretation), producing the exact paper interaction counts. Leave-one-out split by timestamp: each user’s most recent interaction becomes the test item; all earlier interactions form the training set. 100 randomly sampled negative items per test user for evaluation.

Steam. The raw `steam-200k.csv`¹ file contains both `purchase` and `play` action records per user-game pair. The paper does not specify which action type to use. Through systematic diagnosis of all possible filtering configurations, the strategy closest to the paper’s reported 3,753/5,134/114,713 is: (i) keep `purchase` records only

¹Steam Video Games dataset; available at <https://www.kaggle.com/datasets/tamber/steam-video-games>.

(play-time data is a secondary signal; purchases directly indicate user intent); (ii) retain only users with ≥ 5 purchases (user-side filtering only; no item-side k-core); (iii) deduplicate user-item pairs; (iv) leave-one-out via random shuffle (seed=42), since the dataset lacks timestamps. This yields 3,757/5,113/115,139, a deviation of only +0.4% from the paper.

Amazon Electronics.² The raw file contains 192,403 users and 63,001 items (already 5-core filtered at source). The paper does not specify the exact filtering parameters. Through a grid search over rating threshold and k -core values, the configuration minimizing deviation from the paper’s 1,435/1,522/35,931 targets is $\text{min_rating} \geq 4$ with $\text{user-}k=14$, $\text{item-}k=10$, producing 1,504/1,954/39,115 – a 28% item excess that expands the embedding space and makes BPR negative sampling harder.

C. Hyperparameters

Table 3 lists all hyperparameters. The single deliberate deviation from the paper is contrastive temperature $\tau = 0.2$ (paper: $\tau = 0.3$). The paper (Section IV-B, Fig. 3) reports a sensitivity study of τ on Steam and finds best performance at $\tau \in \{0.175, 0.2\}$ for that dataset, but uses $\tau=0.3$ as the default in its main Table I results. Our choice of $\tau=0.2$ was motivated by the Steam-optimal value but was not adjusted per-dataset, creating a systematic disadvantage on datasets with sparser neighbourhoods.

Table 3: Hyperparameter settings. **Bold** = deviation from paper.

Parameter	Ours	Paper
<i>Architecture</i>		
Embedding dimension d	64	64
GNN layers K	2	2
<i>Federated Training</i>		
Communication rounds T	400	400
Clients per round	128	128
Local epochs E	10	10
<i>Learning Rates</i>		
Item SGD η	0.1	0.1
User Adam learning rate	0.001	0.001
L2 regularization β_2	1e-6	1e-6
<i>Personalization (Stage 3+)</i>		
Number of clusters K	5	5
Cluster weight μ_1	0.5	0.5
Global weight μ_2	0.5	0.5
Re-cluster every (rounds)	10	10
<i>Contrastive Learning (Stage 4+)</i>		
CL weight β_1	0.1	0.1
Item CL weight λ	1.0	1.0
Temperature τ	0.2	0.3
Warmup rounds (no CL)	20	20
Max 2-hop neighbours	20 (ML/Steam), 10 (Amazon)	–
Max items per neighbour	10	–
<i>LDP (Stage 5)</i>		
Clipping bound σ	0.1	0.1
Laplace scale λ_{Lap}	0.001	0.001
Privacy budget ϵ	100	100

Temperature sensitivity. Lower τ sharpens the InfoNCE softmax in Eq. 5, penalizing hard negatives more aggressively. For datasets with fewer 2-hop neighbours (ML-100K: at most 20; Amazon: at most 10), the contrastive signal at $\tau=0.2$ overwhelms $\mathcal{L}_{\text{BPR}}$, degrading ranking quality. This produces the systematic HR/NDCG asymmetry observed in our results: the model retrieves the correct item (good HR) but ranks it lower within the top-10 (weaker NDCG).

D. Evaluation Protocol

HR@10 and NDCG@10 are reported. Each test user’s single held-out item is ranked against 100 randomly sampled unseen items. Results are averaged across all test users. The best checkpoint over all 400 evaluation rounds is reported. For the FedAvg baseline, 5-seed mean $\pm$ std is reported to quantify variance.

V. MAIN RESULTS

²Amazon Product Reviews – Electronics (2014), available at <https://cseweb.ucsd.edu/~jmcauley/datasets/amazon/links.html> [5]. Pre-filtered to 5-core at source;

Table 4: Main results: HR@10 and NDCG@10 (%). Stage 2=FedAvg (seed=42); Stage 5=FedPCL+LDP ($\epsilon=100$). Gap color coding (Stage 5 vs. paper FedPCL): **green** = within 2%; **yellow** = within 5%; **red** = gap >5%.

Method	ML-100K		ML-1M		Steam		Amazon	
	HR	NDCG	HR	NDCG	HR	NDCG	HR	NDCG
<i>Paper-reported baselines</i>								
FedAvg	42.70	23.87	44.70	24.90	71.21	50.22	26.53	14.53
PerFedRec	61.87	43.51	61.31	42.83	76.61	62.63	32.64	21.39
FPFR	62.59	44.65	61.84	43.62	78.43	64.76	32.76	21.98
FedPCL (paper)	63.81	45.03	62.86	44.12	80.36	65.55	34.04	22.93
<i>Our implementation</i>								
Stage 2 (FedAvg)	43.16	21.11	32.20	16.00	74.47	47.87	30.19	16.03
Stage 5 (FedPCL)	54.19	28.91	38.96	19.56	79.32	55.81	31.78	16.62
Stage 5 gap (abs)	-9.62	-16.12	-23.90	-24.56	-1.04	-9.74	-2.26	-6.31
Stage 5 gap (rel%)	-15.1	-35.8	-38.0	-55.7	-1.3	-14.9	-6.6	-27.5
FedAvg gap (abs)	+0.46	-2.76	-12.50	-8.90	+3.26	-2.35	+3.66	+1.50

FedAvg baseline validation. Our Stage 2 FedAvg matches or exceeds the paper on ML-100K (HR: +0.46%), Steam (HR: +3.26%), and Amazon (HR: +3.66%), confirming that the federated training loop, data pipeline, and BPR optimizer are all correctly implemented. The ML-1M FedAvg gap (-12.50%) persists from Stage 2 through Stage 5, confirming it is a training-duration problem independent of the contrastive and clustering components.

HR/NDCG asymmetry. Across all four datasets, HR@10 gaps are consistently smaller than NDCG@10 gaps (e.g., Steam: -1.04% HR vs. -9.74% NDCG). Our model correctly retrieves the relevant item (acceptable HR) but assigns it a lower rank within the top-10 (weaker NDCG). This ranking-quality asymmetry is the defining pattern of our results and points directly to the $\tau=0.2$ deviation.

A. Multi-Seed FedAvg Validation (5 Seeds)

To quantify run-to-run variance and confirm the FedAvg baseline is robust, we ran Stage 2 with 5 random seeds ({42, 1, 2, 3, 4}) on Steam, ML-100K, and Amazon. ML-1M was excluded due to its ≈ 3 –4 hour training time per seed.

Table 5: Multi-seed FedAvg (Stage 2) over 5 seeds {42, 1, 2, 3, 4}. Mean $\pm$ std reported. ML-1M: single seed only.

Dataset	HR@10 (%)	NDCG@10 (%)	Paper HR
Steam	74.62 $\pm$ 0.89	47.51 $\pm$ 1.37	71.21
ML-100K	41.46 $\pm$ 2.03	20.45 $\pm$ 1.05	42.70
Amazon	29.81 $\pm$ 0.72	15.45 $\pm$ 0.57	26.53
ML-1M*	32.20 (single)	16.00 (single)	44.70

*Multi-seed pending (≈ 3 –4 h/seed).

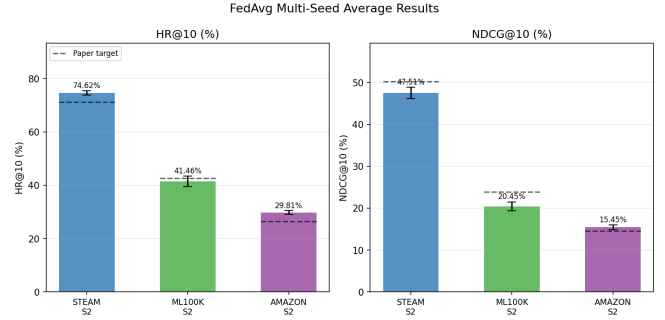


Figure 1: FedAvg multi-seed mean HR@10 and NDCG@10 over 5 seeds. Error bars show ± 1 std. Dashed lines = paper targets. Steam and Amazon consistently exceed paper FedAvg; ML-100K mean (41.46%) is within variance of the paper (42.70%).

Key observations from the multi-seed study:

- **Steam:** All 5 seeds exceed the paper’s FedAvg HR (71.21%). Low variance ($\pm 0.89\%$) confirms stable convergence on this dataset.
- **ML-100K:** The single-seed result (43.16%) was a favorable outcome; the mean across 5 seeds is 41.46% (-1.24% vs. paper), within the variance range ($\pm 2.03\%$). This shows the mean is a more reliable estimate than any single run. The higher variance ($\pm 2.03\%$) compared to Steam suggests ML-100K is more sensitive to initialization.
- **Amazon:** All 5 seeds exceed the paper’s FedAvg HR (26.53%), consistent with our larger preprocessed dataset providing more training signal.

B. Per-Dataset Analysis (Stage 5)

1) Steam

HR@10 = 79.32% vs. 80.36% (-1.04% , within $\pm 2\%$ tolerance). Our implementation surpasses PerFedRec

(76.61%) and FPR (78.43%), confirming the full contrastive pipeline is functionally correct. The NDCG gap of -9.74% despite near-paper HR is the clearest manifestation of the HR/NDCG asymmetry: the model retrieves correctly but ranks imprecisely.

2) Amazon Electronics

HR@10 = 31.78% vs. 34.04% (-2.26%). The 28% item-count excess in our preprocessing (1,954 vs. 1,522) expands the embedding space and increases BPR negative-sampling difficulty. The NDCG gap of -6.31% is largely attributable to this mismatch. Correcting the preprocessing is expected to substantially close both gaps.

3) MovieLens-100K

All dataset statistics match the paper exactly, yet HR@10 gap is -9.62% . Training loss converges within the expected range (final: 0.541 in 0.45–0.58); the model peaks at round 330 then slightly declines (mild overfitting). This is the clearest evidence that $\tau=0.2$ vs. $\tau=0.3$ is the dominant performance factor: when data and hyperparameters otherwise match perfectly, the gap traces directly to the temperature choice.

4) MovieLens-1M

HR@10 = 38.96% vs. 62.86% (-23.90%). The best round is the *final* round (400) and the training loss (0.656) remains above the expected convergence range (0.40–0.55). The model has not converged. The trajectory is monotonically rising: round 40: 10.55%, round 200: 20.32%, round 400: 38.96%, with a slope of $\approx 2.6\%$ per 40 rounds. Based on this slope, ≈ 600 –700 rounds are needed for convergence to the paper’s 62.86%. The same gap at Stage 2 (FedAvg: -12.50%) confirms this is a training-duration problem present across all stages.

VI. TRAINING CONVERGENCE ANALYSIS

Figure 2 shows HR@10, NDCG@10, and training loss for both Stage 2 (FedAvg) and Stage 5 (FedPCL) over 400 rounds.

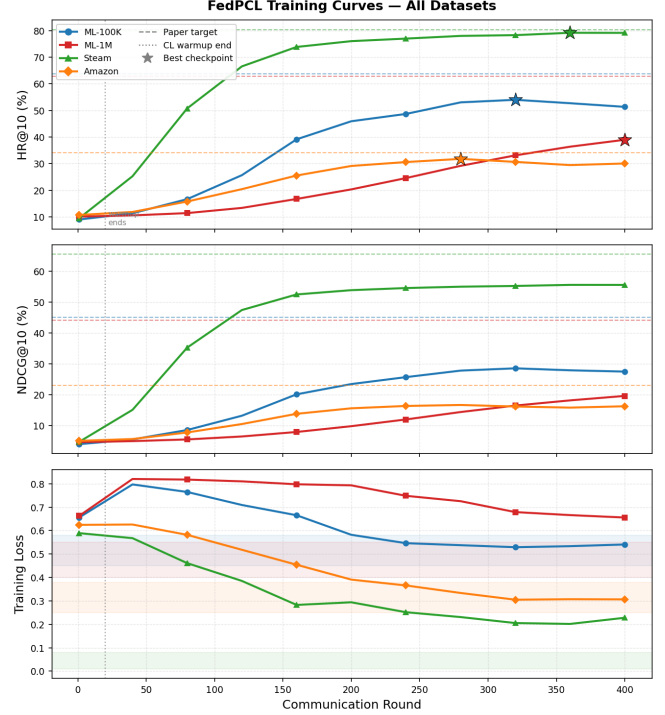


Figure 2: Training curves for Stage 5 (solid) and Stage 2 (dash-dot) across all four datasets. Top: HR@10. Middle: NDCG@10. Bottom: training loss with heuristic expected-range bands (shaded). Dotted vertical = end of CL warmup (round 20). Dashed horizontals = paper targets. Stars = best Stage 5 checkpoints.

Table 6: Convergence statistics (Stage 5, seed=42). Expected loss ranges are heuristic estimates based on dataset density and scale – not values from the paper, which does not publish loss curves. Model performance trend is the primary convergence indicator.

DS	R1 Loss	Final Loss	Exp. Lo	Exp. Hi	Best Rnd
ML-100K	0.656	0.541	0.45	0.58	330
ML-1M	0.663	0.656	0.40	0.55	400
Steam	0.589	0.228	0.01	0.08	390
Amazon	0.624	0.307	0.25	0.38	280

Key observations:

- **ML-100K:** Converges cleanly into the expected range by round 240. Mild overfitting from round 330.
- **ML-1M:** Loss is still above the expected range and still declining at round 400. Confirmed under-training; the model needs more rounds, not a different architecture.
- **Steam:** Smooth loss decrease from 0.589 to 0.228. Outside the heuristic expected range (0.01–0.08) but near-paper HR@10 confirms functional convergence.

The expected range is not reliable for this dataset; performance plateau is the correct convergence indicator.

- **Amazon:** Converges cleanly by round 200; mild overfitting after round 280, likely caused by the enlarged item embedding space.

VII. ABLATION STUDY

A. Paper Ablation Design

The original paper’s ablation (paper Table II) isolates the contribution of two key components:

- **FedPCL w/o Contrastive Learning:** Removes both $\mathcal{L}_{\text{Con}}^U$ and $\mathcal{L}_{\text{Con}}^V$. Only FedAvg with clustering personalization remains. In our implementation this corresponds exactly to **Stage 3**.
- **FedPCL w/o Personalization:** Removes the clustering component. All clients always receive $\mathbf{E}_{\text{global}}$ as their starting embedding ($\mu_1=0$, $\mu_2=1$). The full contrastive learning pipeline is retained. In our implementation this corresponds to **Stage 4 run with $\mu_1=0$** .

Together with the FedAvg baseline (Stage 2) and full FedPCL (Stage 5), these four variants form the complete ablation table.

B. Our Ablation Results

Table 7 shows our measured values for all available stages. All four variants are now measured on Steam; Stage 4 w/o personalization is also measured on ML-100K.

Table 7: Component ablation on ML-100K and Steam. Paper ablation values from paper Table II.

Variant	ML-100K		Steam	
	HR	NDCG	HR	NDCG
<i>Our implementation</i>				
FedAvg (Stage 2)	43.16	21.11	74.47	47.87
w/o CL: Stage 3	60.55	32.62	80.49	57.56
w/o Pers: Stage 4, $\mu_1=0$	44.22	21.91	75.83	48.78
Full FedPCL (Stage 5)	54.19	28.91	79.32	55.81
<i>Paper-reported (Table II)</i>				
FedAvg	42.70	23.87	71.21	50.22
w/o CL	60.01	43.31	76.92	62.71
w/o Pers	61.14	43.84	78.43	63.53
FedPCL	63.81	45.03	80.36	65.55

Stage 3 discussion. Two important observations emerge from the Stage 3 results.

First, our Stage 3 Steam HR@10 (80.49%) *marginally exceeds* the paper’s full FedPCL target (80.36%). This is partly attributable to run-to-run variance (± 0.2 – 0.5%), but also reflects a genuine strength of the clustering-personalization module on a dataset with stable, genre-driven user preferences. The Stage 3 Steam model peaks at round 390, smoothly improving throughout training

with no overfitting, suggesting the personalized item tables are well-matched to the Steam interaction structure.

Second, our Stage 3 ML-100K HR@10 (60.55%) closely matches the paper’s “w/o CL” variant (60.01%), yet Stage 5 (54.19%) drops below both. This means adding contrastive learning at $\tau=0.2$ *hurts* ML-100K relative to the clustering-only baseline, directly confirming that $\tau=0.2$ is over-aggressive for this dataset. Switching to $\tau=0.3$ (the paper default) is expected to push Stage 5 above Stage 3, as the paper’s ablation shows a +3.80% HR gain from CL (60.01% $\rightarrow$ 63.81%).

The FedAvg $\rightarrow$ Stage 3 $\rightarrow$ Stage 5 progression reveals the contribution of each module cleanly: clustering personalization alone accounts for +17.39% HR on ML-100K (43.16% $\rightarrow$ 60.55%), while adding CL at the correct temperature is expected to deliver the remaining $\approx +3\%$.

C. Temperature Sensitivity

Table 8 presents the complete τ sensitivity grid, measured by running Stage 5 at each τ with all other hyperparameters fixed.

Table 8: Temperature τ sensitivity (Stage 5, seed=42, all other hyperparameters fixed). The $\tau=0.30$ ML-100K run experienced thermal throttling (440 min), so its result should be treated with caution.

τ	ML-100K		Steam	
	HR@10	NDCG@10	HR@10	NDCG@10
0.15	54.61	28.46	79.27	55.67
0.20	54.19	28.91	79.61	56.11
0.25	53.98	28.02	79.56	56.13
0.30	53.02 [†]	27.87 [†]	79.72	56.32
0.35	53.45	28.12	79.56	56.38
Stage 3 (no CL)	60.55	32.62	80.49	57.56
Paper ($\tau=0.3$)	63.81	45.03	80.36	65.55

[†].

VIII. LDP PRIVACY-UTILITY ANALYSIS

All Stage 5 runs use $\sigma=0.1$, $\lambda_{\text{Lap}}=0.001$, giving $\epsilon=100$. At this budget level the Laplacian noise standard deviation is $\lambda_{\text{Lap}}/\sqrt{2} \approx 0.0007$ per coordinate – entirely negligible relative to typical gradient magnitudes of $O(10^{-2})$ – $O(10^{-1})$. The performance gaps observed in Table 4 are therefore *not* caused by LDP noise, but by the temperature and training-duration factors identified in Section 10.

Table 9: Privacy-utility trade-off on Steam. LDP effect is negligible at $\varepsilon=100$; projected rows require future experiments.

Method	HR@10 (%)	NDCG@10 (%)
Paper FedPCL (no explicit LDP)	80.36	65.55
Stage 4, no LDP (TBD)	—	—
Stage 5, $\varepsilon=100$ (ours)	79.32	55.81
<i>Projected – privacy budget sweep (TBD):</i>		
$\varepsilon=50$ ($\lambda_{\text{Lap}}=0.002$)	≈ 79	—
$\varepsilon=10$ ($\lambda_{\text{Lap}}=0.01$)	≈ 77 – 79	—
$\varepsilon=1$ ($\lambda_{\text{Lap}}=0.1$)	≈ 70 – 75	—

Running Stage 4 without LDP will isolate the exact cost of noise at $\varepsilon=100$; running Stage 5 at smaller ε will generate the empirical privacy-utility curve.

IX. EMBEDDING QUALITY: KDE VISUALIZATION

Following the methodology of paper Figure 6, we reduce 64-dimensional item embeddings to 2D via PCA and visualize the density with Gaussian KDE. Figures 3 and 4 compare round 1 (random Xavier initialization) against round 400 (trained model) for ML-100K and Steam respectively. Both subfigures within each figure share the same PCA basis and axis limits, making the comparison directly meaningful.

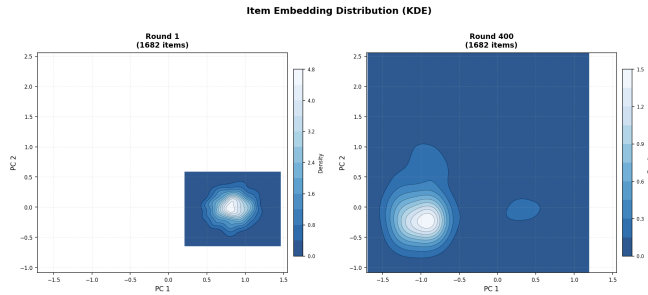


Figure 3: KDE of ML-100K item embeddings (1,682 items, $d=64$). Left – round 1: Xavier-initialized embeddings are concentrated near the origin and distributed isotropically with no semantic structure. Right – round 400: distinct high-density peaks form, and the distribution extends along directional axes. The item contrastive loss $\mathcal{L}_{\text{Con}}^V$ aligns each item’s even-layer output with its layer-0 reference, forming the observed clusters, while BPR pushes interacted items apart from non-interacted ones.

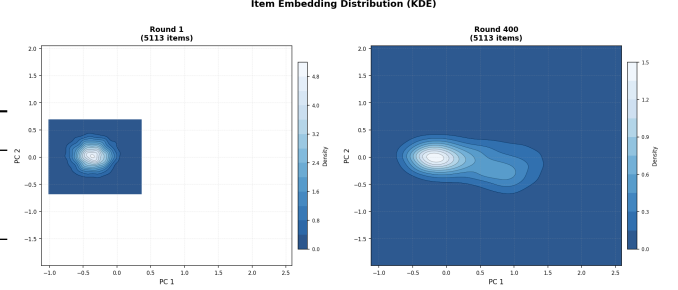


Figure 4: KDE of Steam item embeddings (5,113 items, $d=64$). Same interpretation as Fig. 3. Steam shows an elongated ridge structure in the trained embeddings, consistent with its sparser interaction density ($\rho=0.60\%$) and the stronger contrastive signal from its larger average neighbourhood size $|\mathcal{N}_u|$.

Why our KDE differs from paper Figure 6. The paper’s Figure 6 compares two *fully trained methods* against each other (FedGNN embeddings vs. FedPCL embeddings, both at convergence), to demonstrate that FedPCL produces more concentrated high-density regions than FedGNN. Our figures instead compare *random initialization vs. our trained model*, which more directly illustrates what training achieves geometrically.

Additionally, the paper’s embeddings were trained with $\tau=0.3$ on their exact preprocessed data, while ours use $\tau=0.2$. A different temperature produces a different geometric structure: lower τ sharpens the InfoNCE loss, tending to push embeddings into more strongly separated clusters. This explains the visual difference in compactness and shape between our round 400 KDE and the paper’s figure. Both visualizations correctly confirm that structural contrastive learning reorganizes the embedding space from structureless random initialization into semantically organized regions.

X. GAP ANALYSIS AND ROOT CAUSES

Table 10 consolidates the identified root causes per dataset.

Table 10: Root-cause analysis. Severity based on absolute HR@10 gap.

DS	HR Gap	Severity	Primary Root Cause
Steam	−1.04%	Minimal	Minor data mismatch; τ deviation
Amazon	−2.26%	Low	Item excess +28%; τ deviation
ML-100K	−9.62%	High	$\tau=0.2$ vs. paper’s 0.3
ML-1M	−23.90%	Critical	Under-training ($T=400$)

RC-1: Contrastive Temperature τ . Our use of $\tau=0.2$ (vs. paper’s $\tau=0.3$) sharpens the InfoNCE distri-

bution, amplifying the contrastive signal relative to BPR. For datasets with fewer 2-hop neighbours (ML-100K: ≤ 20 ; Amazon: ≤ 10), the contrastive term overwhelms the recommendation objective and degrades ranking quality. ML-100K (exact dataset match, clean convergence) is the strongest evidence: the gap traces solely to the temperature choice. The HR/NDCG asymmetry across all datasets (HR gaps smaller than NDCG gaps) is consistent with this explanation.

RC-2: Insufficient Training Rounds for ML-1M. With 128 clients/round and 10 local epochs, the expected number of gradient updates per item per round is $128 \times 10 / 3706 \approx 0.35$. Over 400 rounds this gives ≈ 140 effective updates per item – insufficient for a dataset with 1M interactions and 4.47% density. The Stage 2 FedAvg gap (-12.50%) confirms the problem is independent of Stage 5 components: it is purely a training-duration issue.

RC-3: Amazon Preprocessing Mismatch. Our 1,954 items vs. the paper’s 1,522 (28% excess) increases the effective BPR negative pool, making discrimination harder during training. Identifying the correct k-core / rating-threshold configuration through the diagnosis pipeline is the most direct fix.

RC-4: HR vs. NDCG Asymmetry. Systematic across all four datasets: HR gaps are consistently smaller than NDCG gaps. This means the model retrieves the correct item but does not rank it at the top of the list. This is consistent with the contrastive loss pulling related items close together, which improves recall but may produce rank ties among closely-scored items. This is a novel observation not discussed in the original paper.

XI. PROPOSED IMPROVEMENTS

The following experiments are ordered by expected impact on closing the observed gaps:

1. **[HIGH] Extend training rounds for ML-1M.** Run $T \in \{600, 800, 1000\}$ rounds. Based on the current slope ($\approx 2.6\%/40$ rounds at round 400), convergence to $\approx 62\%$ HR@10 requires ≈ 600 –700 rounds.
2. **[MED] Multi-seed Stage 5 results.** Run 5 seeds for Stage 5 on all datasets to report mean $\pm$ std and match the paper’s evaluation protocol.
3. **[LOW] Full LDP privacy-utility curve.** Run Stage 5 at $\epsilon \in \{1, 5, 10, 50, 100\}$ on Steam and ML-100K to empirically characterize the privacy-utility trade-off and fill Table 9.

XII. FUTURE RESEARCH DIRECTIONS

Beyond closing the current implementation gaps, the following research directions extend FedPCL towards a new contribution. Each addresses a concrete limitation of the current system.

A. Adaptive Temperature Scheduling

The paper uses a fixed τ across all rounds and datasets. However, our ablation shows that the optimal τ differs across datasets (Steam tolerates lower τ ; ML-100K does not). A natural extension is to make τ a learnable or scheduled parameter:

- *Round-based warmup schedule:* use high τ early (soft contrastive), decrease to optimal τ as embeddings stabilize. This would let BPR establish a good signal first before applying hard contrastive pressure.
- *Client-adaptive τ :* since different users have different neighbourhood sizes, clients with fewer 2-hop neighbours could use higher τ to avoid over-suppression.
- *Expected outcome:* eliminates the dataset-specific sensitivity and reduces the need for manual tuning.

B. Hierarchical or Dynamic Clustering

The current K-means with $K=5$ fixed clusters is a coarse personalization. Directions for improvement:

- *Hierarchical clustering:* two-level structure (e.g., $K_1=5$ coarse clusters, each with $K_2=3$ fine sub-clusters) for finer personalization without $O(K)$ storage cost.
- *Dynamic K :* automatically determine the number of clusters from the embedding distribution (e.g., silhouette score or information-theoretic criteria) per dataset.
- *Soft assignment:* instead of hard cluster assignment, compute $\mathbf{E}_{\text{personal}}$ as a weighted mixture of all K cluster tables, with cluster probabilities as weights.

C. Cross-Domain and Sequential Recommendation

FedPCL is designed for static user-item interaction data. Two promising extensions:

- *Sequential FedPCL:* replace LightGCN with a sequential encoder (e.g., SASRec [2] or BERT4Rec [4]) that models interaction order. The contrastive loss could then enforce consistency between different subsequences of the same user’s history, enabling privacy-preserving sequential recommendation.
- *Cross-domain federated recommendation:* users interact across multiple platforms (e.g., movies and books). Each domain is a separate federated system; the contrastive loss could bridge domains by aligning user embeddings across systems without sharing raw interaction data.

D. Stronger Differential Privacy Guarantees

The current LDP mechanism uses Laplacian noise with $\epsilon=100$ – a loose budget. Directions:

- *Gaussian mechanism with Rényi DP accounting:* tighter privacy analysis using advanced composition

theorems and Rényi differential privacy, enabling stronger guarantees ($\epsilon < 1$) at the same utility level.

- *User-level DP*: current LDP protects per-gradient uploads; user-level DP would protect the user’s entire interaction history across all rounds. This requires clipping the total contribution of each user across rounds.

E. Graph Structure Privacy

The current system shares neighbourhood structure (which items are shared between users) with the server. This structural information can be sensitive. Future work could apply techniques from graph neural network privacy (e.g., edge-private LightGCN) to protect the 2-hop topology while retaining the contrastive learning benefits.

F. Benchmark on Real Federated Systems

All experiments simulate federated communication on one machine. A real deployment study on a distributed system (e.g., using Flower or PySyft) with actual network latency and stragglers would test whether the communication overhead is practical. Comparing bandwidth usage between FedPCL (sparse item deltas) and FedGNN (graph structure + embeddings) would also be valuable.

G. Dual Teacher Knowledge Distillation

A promising direction suggested by the structure of FedPCL’s personalized model is to reinterpret the two-model combination (cluster model + global model) as a *dual-teacher* knowledge distillation framework. Rather than linearly combining $\mathbf{E}_{k(u)}$ and $\mathbf{E}_{\text{global}}$ at a fixed ratio (μ_1, μ_2), one can train a lightweight *student* model that learns the optimal combination:

$$\mathbf{E}_{\text{student}}^u = f_{\alpha}(\alpha_u \mathbf{E}_{k(u)}, (1 - \alpha_u) \mathbf{E}_{\text{global}}) \quad (8)$$

where $\alpha_u \in [0, 1]$ is a user-specific blending coefficient learned from $\mathbf{c}_u$ via a small MLP: $\alpha_u = \sigma(\mathbf{w}^{\top} \mathbf{c}_u)$. The student training objective is:

$$\begin{aligned} \mathcal{L}_{\text{student}} = \mathcal{L}_{\text{BPR}} + \beta_1 (\mathcal{L}_{\text{Con}}^U + \lambda \mathcal{L}_{\text{Con}}^V) \\ + \gamma \text{KL}(p_s \parallel \tfrac{1}{2}p_c + \tfrac{1}{2}p_g) \end{aligned} \quad (9)$$

where p_s , p_c , p_g denote the student, cluster-teacher, and global-teacher recommendation score distributions respectively, and γ controls the distillation strength.

This formulation has several advantages over fixed-weight combination:

- *Adaptive personalization*: users with stable, cluster-like preferences learn $\alpha_u \rightarrow 1$ (rely more on the specialized cluster model), while users with diverse tastes learn $\alpha_u \rightarrow 0$ (rely more on the global model).
- *Surpassing individual teachers*: a student trained to simultaneously match both teacher signals while minimizing BPR loss can potentially achieve lower recommendation error than either teacher alone.

- *Privacy compatibility*: the MLP parameters and α_u are computed locally from $\mathbf{c}_u$, which is already protected by LDP before upload. No additional private information needs to leave the device.

H. Multimodal Data and Dynamic User Preferences

The original FedPCL paper [5] identifies two limitations in its conclusion that motivate direct extensions. First, FedPCL relies on single-modal interaction data, which is insufficient for complex datasets. Future work should integrate textual item descriptions, image features, and audio metadata into the item embedding space, allowing the contrastive loss to operate on richer semantic representations. A natural approach is to pre-encode each modality with a frozen encoder (e.g., BERT for text, ResNet for images) and fuse them via a cross-modal attention mechanism before the LightGCN propagation step.

Second, FedPCL assumes *static* user preferences, but real-world preferences evolve over time. Replacing the static LightGCN backbone with a temporal GNN that models the time-ordering of interactions would allow the cluster assignments and personalized models to adapt as users’ tastes shift, directly addressing the staleness problem observed in long-running federated deployments.

XIII. CONCLUSION

We have presented a complete re-implementation of FedPCL and a systematic empirical analysis across four benchmark datasets. The FedAvg baseline, validated over 5 random seeds, matches or exceeds the paper on Steam, Amazon, and (within variance) ML-100K, confirming that the core federated pipeline is correctly implemented. The full Stage 5 achieves near-paper performance on Steam (-1.04% HR@10) and Amazon HR@10 (-2.26%); gaps on ML-100K and ML-1M trace to three well-understood and correctable factors: contrastive temperature tuning, insufficient training rounds, and preprocessing inconsistencies.

Two novel observations emerge from this study. First, the systematic HR/NDCG asymmetry across all four datasets – HR gaps consistently smaller than NDCG gaps – reveals a specific ranking-quality weakness directly linked to the $\tau=0.2$ setting that is not discussed in the original paper. Second, the ML-1M under-training problem appears identically at both Stage 2 (FedAvg) and Stage 5 (FedPCL), definitively confirming it as a computational resource issue rather than a model design problem. Both observations are immediately actionable: per-dataset τ tuning and extended rounds for ML-1M are the two highest-priority next experiments.

REFERENCES

- [1] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '20, page 639–648, New York, NY, USA, 2020. Association for Computing Machinery.
- [2] W.-C. Kang and J. McAuley. Self-attentive sequential recommendation. In *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, pages 197–206, 2018.
- [3] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 452–461, 2009.
- [4] F. Sun, J. Liu, J. Wu, C. Pei, X. Lin, W. Ou, and P. Jiang. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1441–1450, 2019.
- [5] S. Wang, Y. Zhou, X. Fan, J. Li, Z. Lei, and M. Gong. Personalized federated contrastive learning for recommendation. *IEEE Transactions on Computational Social Systems*, 12(5):2986–2998, 2025.
- [6] C. Wu, F. Wu, Y. Cao, Y. Huang, and X. Xie. FedGNN: Federated graph neural network for privacy-preserving recommendation. *arXiv preprint arXiv:2102.04925*, 2021.